CF SRAM INTEGRATION

As a Physical Design Lead of student tapeout organization, SiliconJackets, I was responsible for integrating the new SRAM macro into our current tapeout flow.

– Jiho Jun

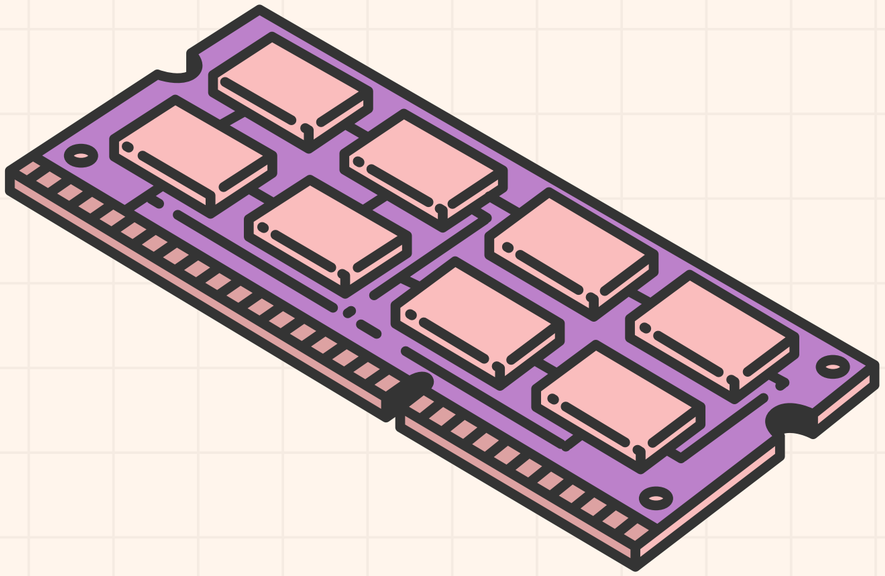
chipfoundry

PROBLEM

Up to this point, SiliconJackets' **RTL-to-GDS** flow has integrated a Cadence-provided SRAM model, which has allowed us to validate the implementation pipeline and establish a baseline physical design.

However, production tape-outs typically rely on foundry-provided memory macros that are characterized across multiple process, voltage, and temperature (**PVT**) corners. The ChipFoundry SRAM provides additional physical and timing views, enabling more realistic integration and expanding the flow's capability toward **MMMC** (Multi-Mode Multi-Corner) timing analysis.

Transitioning to a foundry-style SRAM macro aligns the flow more closely with industry-standard methodology and enables future work on corner-aware timing analysis, signoff robustness, and scalability.



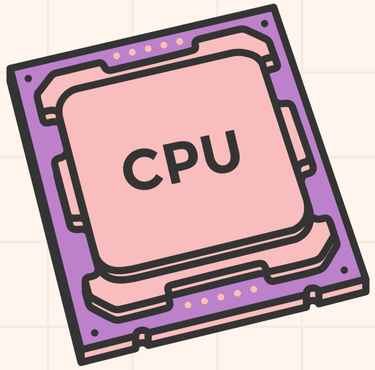
Commercial 32KB SRAM

Available

High-performance 32KB hard IP SRAM macro with Wishbone interface, designed for efficient SoC integration using SkyWater SKY130 process.

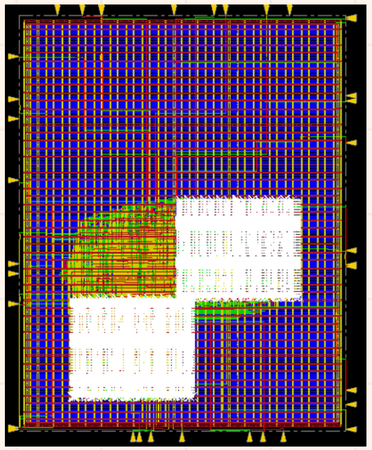
Name	Version	Price	Release Date
CF_SRAM_8192x32	v1.0.3	$2500	August 11, 2025
Certification	**Foundry**	**Node**	**Technology**
Vendor Certified	SkyWater	130nm	sky130
Type	**Maturity**	**Bus Interfaces**	**License**
Hard	Production	WB	Commercial

MAIN TASKS



MACRO INTEGRATION

- Integrate the ChipFoundry SRAM LEF, Liberty (.lib), and any views into the flow.
- Update setup.yaml and configuration files to reference the new memory views.



PHYSICAL IMPLEMENTATION VALIDATION

- Make sure there are no DRC, LVS, PG connectivity violations.
- Timing validation – make sure DO (data output) pins are visible in the timing report.

BRANCH & DOCUMENTATION

- Create a dedicated Git branch for SRAM integration.
- Document required flow and configuration changes.

TIMING VALIDATION ISSUE

Debugging pipeline: run the flow → design file is generated → upload on Cadence software → run tests

In this case,

- ran timing report
- no SRAM DO pins were appeared
- ran “unconstrained” timing report (no timing condition setup/hold defined by designer; includes all the paths, even invalid paths)
- DO corresponding cell is latch (!) not flipflop.
  - Big problem since STA is all about when data arrives but latch is not edge sensitive like flipflops.
  - capture happens ANYWHERE in a time window.
  - This is RTL problem most of the time.
- Fixed RTL
- Ran timing report again
- Works

```
Path 10: VIOLATED (-2.683 ns) Setup Check with Pin u_resbuf_internal_buffer_reg[61]/CK-
              View: tt_v1.8_25C_Nominal_25_func
             Group: core_clock
        Startpoint: (R) sram_A/CLKin
             Clock: (R) core_clock
          Endpoint: (F) u_resbuf_internal_buffer_reg[61]/D
             Clock: (R) core_clock

                     Capture       Launch
        Clock Edge:+   1.000        0.000
       Src Latency:+  -0.335       -0.335
       Net Latency:+   0.491 (P)    0.354 (P)
           Arrival:=   1.156        0.019

             Setup:-   0.166
       Uncertainty:-   0.250
       Cppr Adjust:+   0.000
     Required Time:=   0.739
      Launch Clock:=   0.019
         Data Path:+   3.403
             Slack:=  -2.683

#-----------------------------------------------------------------------------------------------------------------
# Timing Point                          Flags  Arc           Edge  Cell            Fanout  Trans   Delay  Arrival
#                                                                                            (ns)    (ns)     (ns)
#-----------------------------------------------------------------------------------------------------------------
  sram_A/CLKin                          -      CLKin         R     (arrival)            7  0.138       -    0.019
  sram_A/DO[0]                          <<<    CLKin->DO[0]  R     CF_SRAM_1024x32      2  0.138   2.352    2.371
  prects_FE_RC_339_0/Y                  -      B->Y          F     NAND2X4              1  0.046   0.046    2.417
  prects_FE_RC_337_0/Y                  -      A->Y          R     INVX4                1  0.059   0.051    2.468
  prects_FE_RC_44_0/Y                   -      B->Y          F     NAND3X4              2  0.036   0.078    2.546
  g29937/Y                              -      B->Y          R     NAND2X4              1  0.068   0.057    2.603
  g29939/Y                              -      B->Y          F     NAND2X4              1  0.045   0.051    2.654
  g29938/Y                              -      B->Y          R     NAND2X4              1  0.037   0.070    2.724
  g29829/Y                              -      D->Y          F     NAND4X4              1  0.070   0.082    2.805
  cts_FE_RC_150_0/Y                     -      C->Y          R     NAND3X4              1  0.081   0.079    2.884
  cts_FE_RC_151_0/Y                     -      A->Y          F     CLKINVX8             8  0.067   0.070    2.954
  prects_FE_RC_335_0/Y                  -      B->Y          R     NAND2X1              1  0.050   0.066    3.020
  prects_FE_RC_331_0/Y                  -      B->Y          F     NAND2X2              1  0.058   0.063    3.083
  prects_FE_RC_330_0/Y                  -      B->Y          R     NAND2X4              2  0.051   0.068    3.151
  prects_FE_RC_272_0/Y                  -      A->Y          F     NAND2X4              4  0.062   0.073    3.224
  prects_FE_DBTC14_n_6106/Y             -      A->Y          R     INVX4                1  0.060   0.051    3.275
  g29736/Y                              -      B->Y          F     NAND2X4              2  0.036   0.052    3.327
  prects_FE_RC_235_0/Y                  -      C->Y          R     NAND3X2              1  0.046   0.052    3.379
  g29734/Y                              -      B->Y          F     NAND2X2              1  0.049   0.044    3.423
  u_resbuf_internal_buffer_reg[61]/D    -      D             F     SDFFX1               1  0.030   0.000    3.423
#-----------------------------------------------------------------------------------------------------------------
```

Constrained timing report after fixing RTL → DO pins appear now

```
// cursed latch inference
always_latch begin
    if (rst_i) begin
      read_data_curr <= '0;
      read_data_next <= '0;
    end else begin
        if (!r_ptr[0])
            read_data_curr <= r_data;
        else
            read_data_next <= r_data;
    end
end

// latch carry_reg to carry_out at S_ADD state
always_latch begin
    if (rst_i || state == S_READ) begin
        carry_reg <= 1'b0;
    end else begin
        if (state == S_ADD)
            carry_reg <= carry_out;
    end
end
```

controller.sv that contained latch logic

PG (POWER & GROUND) CONNECTIVITY ISSUE

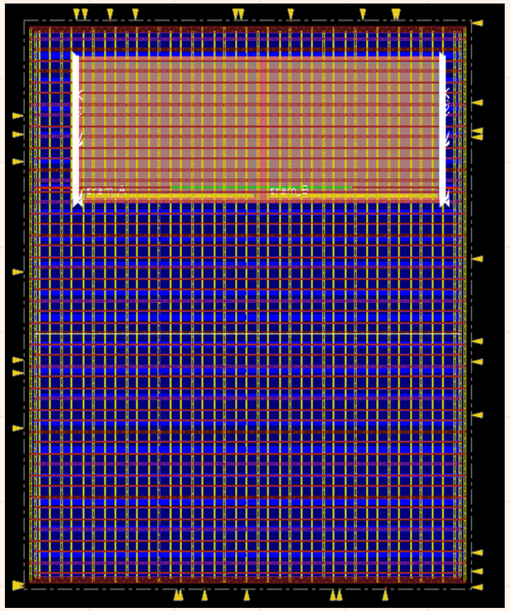
Also wanna make sure there is no connectivity issue in the chip.

In this case,

- ran connectivity report
- terminals are not connected (bunch of white markers)
- check locations of the violations
  - Bunch of unconnected pins between 2 SRAMs.
- Find corresponding powerplanning script in the flow repository
  - Added specific Innovus command
- Ran flow again (only until the floorplan stage; we only care about powerplanning)
- Ran connectivity report again
- Works

```
@innovus 157> check_connectivity -type all -geometry_connect -error 1000 -warning 50
VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Mon Feb 23 00:16:27 2026

Design Name: top_lvl
Database Units: 1000
Design Boundary: (0.0000, 0.0000) (1025.8000, 1301.8000)
Error Limit = 1000; Warning Limit = 50
Check all nets
Net VSS: has an unconnected terminal, has special routes with opens, has regular routing with opens.
Net VDD: has an unconnected terminal, has special routes with opens, has regular routing with opens, dangling Wire.

Begin Summary
    22 Problem(s) (IMPVFC-96): Terminal(s) are not connected.
    2 Problem(s) (IMPVFC-200): Special Wires: Pieces of the net are not connected together.
    455 Problem(s) (IMPVFC-92): Pieces of the net are not connected together.
    4 Problem(s) (IMPVFC-94): The net has dangling wire(s).
    483 total info(s) created.
End Summary

End Time: Mon Feb 23 00:16:28 2026
Time Elapsed: 0:00:01.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 483 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.2  MEM: 0.625M)
```

Report showing that some terminals are not connected

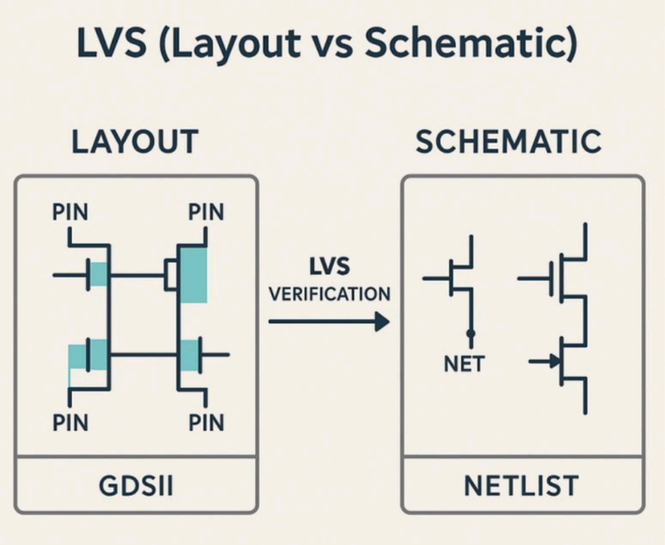


floorplan.enc showing that the violation between the SRAMs are gone

LVS FAILURE

Layout Versus Schematic (LVS) is a critical sign-off verification step in VLSI design that ensures the physical layout (GDSII) is electrically identical to the circuit schematic or netlist.

- As of now, physical implementation should be complete.
- My expectation is that LVS shows MATCH between schematics and layout.
- It doesn't – the tool thought this circuit has extra/missing connections
- Natural suspicion is that adding SRAM might've messed up the abstraction.
- Turns out:
  - It was treating internal signals as external pins
  - It was also treating power (VDD/VSS) like normal signals
- Had to do some overrides (dangerous) but it was obvious tool was acting stupid.
  - Ignore internal SRAM pins
  - Treat power as global (not regular signals)
- Ran again
- Matched




```
#######################################################################
#########################################
#####
#####                                        Pegasus LVS COMPARISON
#####
#####  Version                        :   24.14-s012
#####  NVN Run Start                  :   Sat Mar 21 22:15:41 2026
#####  Comparison Report File         :   top_lvl.lvsrpt.cls
#####  Top Cell                       :   top_lvl  <vs>  top_lvl
#####
#####  Run Result                     :   MATCH
#####
#####  Run Summary                    :   [INFO]  Cells Have Been
Blackboxed
#####                                 :   [INFO]  Some Sections Have
Been Truncated
#####                                 :   [INFO]  Unconnected
Corresponding Top Cell Pin(s) Exists
#####
#####  Layout Design                  :   ./postroute_outputs/golden/
golden_top_lvl.gds (GDSII)
#####  Layout File                    :   /nethome/jjun49/Physical-
Design-Onboarding/RTL2GDS/sapr/LVS_signoff/top_lvl.net (cdl)
#####  Schematic File                 :   ./postroute_outputs/golden/
golden_top_lvl_lvs.cdl (cdl)
#####  Rules File                     :   /nethome/jjun49/Physical-
Design-Onboarding/RTL2GDS/sapr/./resources/cadence_sky130/
sky130_release_0.0.9/Sky130_LVS/sky130.lvs.pvl
#####                                 :   /nethome/jjun49/Physical-
Design-Onboarding/RTL2GDS/sapr/./resources/cadence_sky130/
sky130_release_0.0.9/Sky130_LVS/black_box.pvl
#####                                 :   /nethome/jjun49/Physical-
Design-Onboarding/RTL2GDS/sapr/./resources/cadence_sky130/
sky130_release_0.0.9/Sky130_LVS/lvs_overrides.pvl
#####  Pin Swap File                  :   top_lvl.lvsrpt.cps
#####
#####  NVN CPU Time                   :   0h 0m 1s - (1s)
#####  NVN Exec Time                  :   0h 0m 1s - (1s)
#####  NVN Peak Memory Usage          :   0.00MB
#####
#######################################################################
#########################################
```

LVS report showing MATCH

ECE SKILLS GAINED

SYSTEMATIC DEBUGGING METHODOLOGY

- Broke problems down stage-by-stage (timing → power → verification)
- Isolated root causes instead of guessing
- Verified fixes step-by-step

MULTI-LEVEL SYSTEM THINKING

- Understood how issues propagate across:
  - RTL
  - Physical design
  - Verification
- Learned that fixing one layer can expose problems in another

TOOL INTERPRETATION & PROBLEM FRAMING

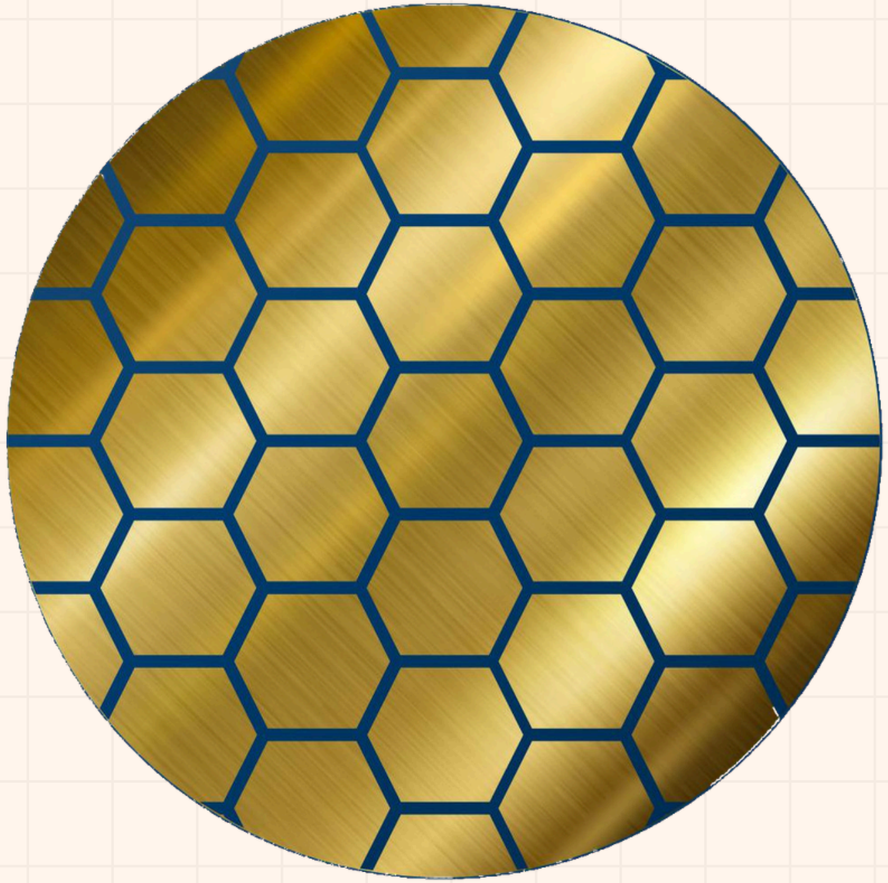
- Learned to interpret tool outputs critically (not just trust them)
- Identified when errors came from:
  - actual design issues
  - vs tool misinterpretation
- Reframed problems to match how tools analyze designs

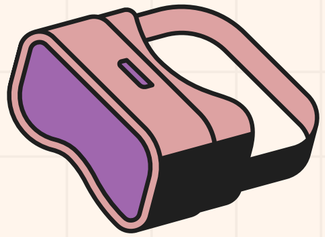
FINAL THOUGHTS

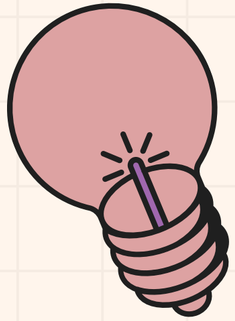
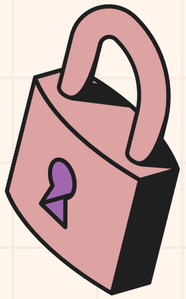
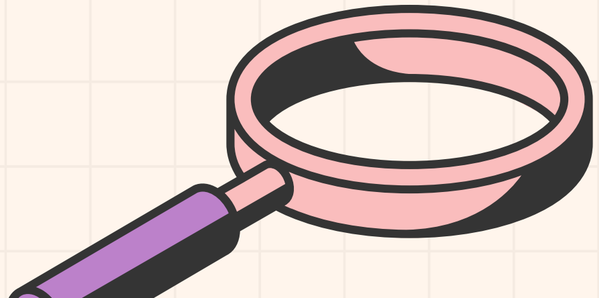
This project was one of the most valuable hands-on experiences I've had because it showed me the gap between a design that looks correct at a high level and one that is actually implementation-ready. I initially approached this as a simple integration task, but it turned into a multi-stage debugging process involving timing, power connectivity, and verification, which gave me a much deeper understanding of how real chip design works.

It significantly strengthened my interest in physical design, especially backend implementation and signoff, because I enjoyed working through practical issues and understanding how different parts of the flow interact. It also made me more interested in debugging and analysis, since a lot of the work involved interpreting tool behavior and tracing issues across multiple abstraction levels.

If I were to continue this project, I would focus on extending the design to a more complete and signoff-clean flow, and explore more complex macro-based designs to further develop my understanding of timing closure and large-scale integration challenges.







THANK YOU